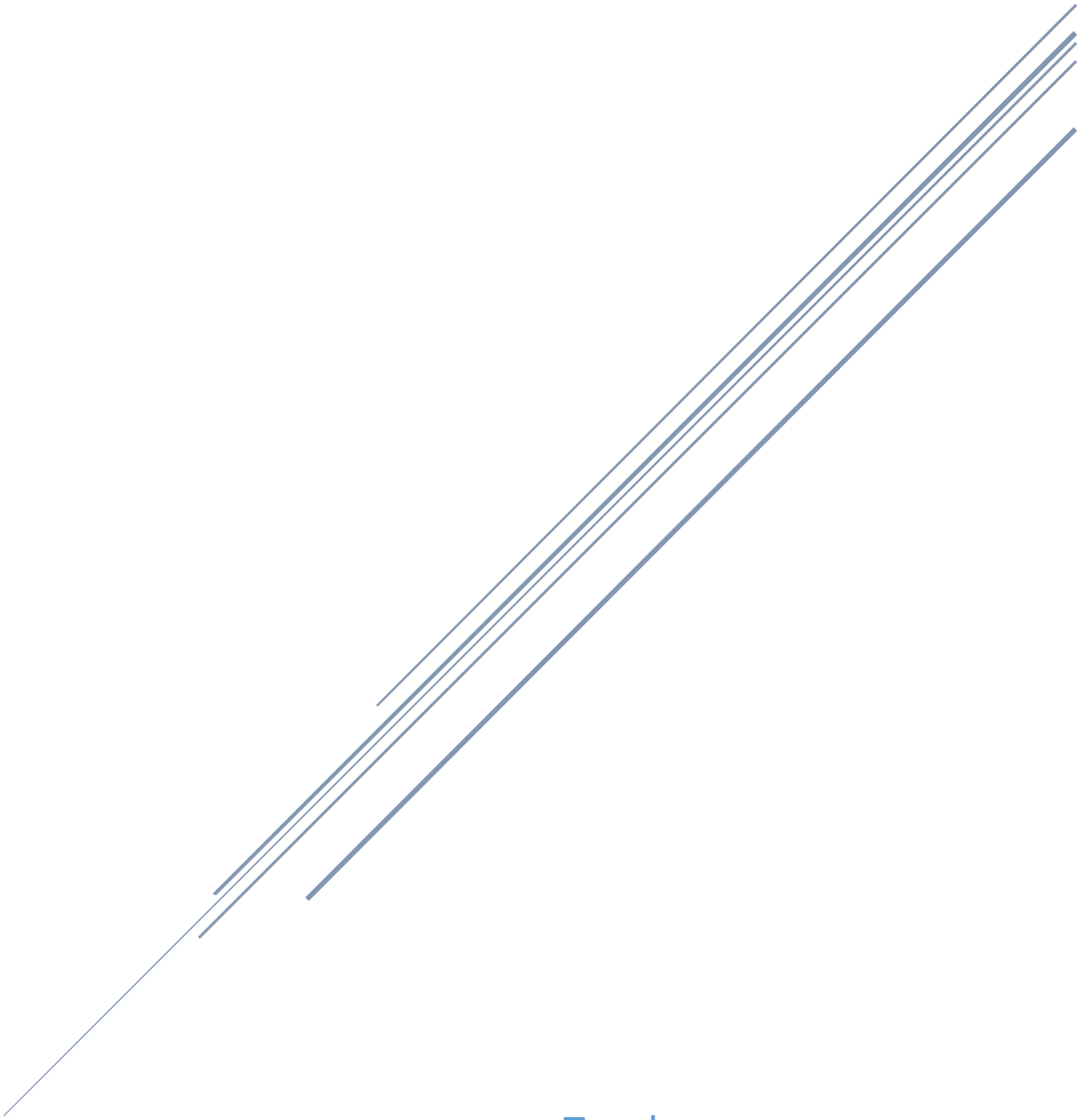
CODE AND SUMMARY OF CLASS 14

Sahir Ahmed Sheikh
Saturday (2 – 5)



Teachers:
Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 14 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a detailed summary of Class 14 of Quarter 3, held on Saturday from 2 to 5 PM, led by Sir Ali Aftab Sheikh and Sir Muhammad Bilal. After a three-week break due to weather-related cancellations, the session resumed, focusing on AI agents and their practical applications. The class covered **Chainlit**, **Chatbot development**, **Streaming**, and **Tools**, building on previous topics like Open AI Agent SDK, Gemini integration, and Large Language Models (LLMs). In today's class, Sir Ameen also joined and provided an in-depth explanation of **tool calling**. The session included theoretical discussions, coding demonstrations, and motivational guidance to encourage self-directed learning and resilience for Quarter 4's advanced topics.

The class addressed the following key objectives:

- Reviewing progress and assignments from the previous class, emphasizing Agent and Runner classes.
- Introducing Chainlit, Chatbot development, Streaming, Tools and Tool calling.
- Encouraging students to focus on their own pace, avoid comparing with other classes, and prepare for Quarter 4's self-directed learning.
- Emphasizing the importance of assignment completion and preparation for the upcoming test.
- Highlighting the importance of practical coding, observability, and professional development practices for industry relevance.

Students were reminded of the limited classes remaining in Quarter 3, the strict policy on assignment completion, and the need to push code to GitHub. The session also included motivational advice to take the course seriously and prepare for practical, hands-on test questions.

Topics Covered

1. Class Updates and Quarter 3 Progress

- **Quarter 3 Overview**: Quarter 3 is nearing its end, with only a few classes remaining. The environment was described as manageable, but Quarter 4 was introduced as significantly more challenging, requiring immediate preparation.
- **Test Updates**: A test is scheduled within August, with practical questions requiring hands-on coding experience. Students were advised to re view documentation and run code to prepare effectively.
- **Assignment Policy**: A new policy was introduced to check for incomplete assignments rather than completed ones. Students with incomplete assignments risk elimination from the course unless they have a genuine reason.
- **Motivational Advice**: Students were urged to avoid procrastination, practice coding regularly, and use tools like NotebookLM for efficient learning through summaries, audio, and interactive modes.

2. Revision Of AI Agent Concepts:

The session continued exploring advanced Agentic AI topics, emphasizing efficient communication, lightweight models, and infrastructure for scaling agents. Key points included:

- **Swarm**: A system that allows multiple agents to collaborate on a single task, dividing responsibilities to achieve results faster and more efficiently.
- **UV**: Acts as a secure execution environment where agents can run tools safely, ensuring controlled and reliable operations.
- **OpenRouter**: A routing system that connects agents to different models or tools, helping them choose the best path or model for completing a given task.
- **Light LLMs (Lightweight Language Models):** Smaller, faster AI models optimized for specific tasks, offering quick responses with low

computational costs — ideal for tools and fast decision-making within agents.

3. AI Architecture: Chainlit, Chatbot, Streaming, Tools, & Tool Calling

3.1 Chainlit

***Definition and Purpose**: Chainlit is an open-source Python framework designed to simplify the development of AI-based applications, particularly conversational interfaces like chatbots. It integrates frontend and backend logic, allowing developers to create interactive, user-friendly applications without needing to write separate frontend code (e.g., in JavaScript or HTML). Chainlit provides a web-based, chatbot-like user interface (UI) where users can interact with an AI agent, and responses are displayed in a conversational format.*

Key Features:

- **Integrated UI:** Chainlit automatically generates a web-based chat interface, eliminating the need for developers to design a frontend from scratch. This is particularly useful for rapid prototyping and deployment of AI applications.
- **Asynchronous Support:** Chainlit supports asynchronous programming, enabling efficient handling of real-time interactions and streaming responses.
- **Customizability**: Developers can customize the UI and backend logic to suit specific use cases, such as integrating with Large Language Models (LLMs) like Gemini or Open AI.
- **Ease of Use:** It integrates seamlessly with Python libraries like openai-agents and python-dotenv, making it accessible for Python developers.

How It Works:

- Chainlit uses decorators like @on_chat_start and @on_message to define the behavior of the application when a chat session begins or when a user sends a message.
- It maintains a session state using user_session to store configurations, agents, and chat history, ensuring context-aware conversations.
- The framework connects to an LLM via an API (e.g., Gemini API) to process user inputs and generate responses, which are then rendered in the UI.

Practical Application:

- In today's class, students used Chainlit to build a chatbot that interacts with the Gemini API. The framework handles the UI, while the backend logic processes user queries through an AI agent, making it ideal for creating conversational AI applications.
- **Example:** A student can create a project using Chainlit to build a customer support bot that answers queries about a product by fetching responses from an LLM.

Why Its Important:

- Chainlit reduces development time by providing a ready-to-use UI, allowing developers to focus on backend logic and AI integration.
- It's highly relevant for Quarter 4's focus on advanced AI applications, as it enables students to build industry-standard conversational interfaces.

3.2 Chatbot

***Definition and Purpose**: A chatbot is an AI tool that interacts with users through a text-based interface. In Class 14, it was used with Chainlit to build an assistant that processes user input and responds using an LLM like Gemini.*

Key Features:

- **User Interface:** Provided by Chainlit, the UI allows users to type messages and view responses in a chat-like format.
- **AI Agent:** AI Agent: The backend logic, implemented using the openai-agents library, defines the chatbot's behavior and instructions (e.g., "You are a helpful assistant").
- **LLM Integration**: The chatbot connects to an LLM to process user inputs and generate responses. For example, it uses the Gemini API to fetch answers.
- **Chat History:** The chatbot maintains a history of user and assistant messages to provide context for ongoing conversations, stored in user_session.

How It Works:

- When a user starts a session, Chainlit's @on_chat_start decorator initializes the agent and configuration (e.g., model, API key).
- When a user sends a message, the @on_message decorator triggers the agent to process the input, append it to the chat history, and query the LLM for a response.
- The response is sent back to the Chainlit UI, displayed as a message from the assistant.

Practical Application:

- In today's class, students built a chatbot using Chainlit and the openai-agents library. The chatbot was configured to answer general questions by querying the Gemini API and displaying responses in the Chainlit UI.
- The following example demonstrates how to build a simple AI chatbot using Chainlit and the openai-agents library.
- It connects to the Gemini API to generate responses based on user input in a conversational interface.

Example:

```python
from dotenv import load_dotenv
from chainlit import on_message, on_chat_start, user_session
from openai_agents import Agent, Runner
import os
from typing import cast

load_dotenv()
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
if not GEMINI_API_KEY:
    raise ValueError("Gemini API key is not set. Please ensure it's defined in your .env")

@on_chat_start
async def start():
    external_client = AsyncOpenAI(base_url="https://api.gemini.com/v1", api_key=GEMINI_API_KEY)
    run_config = {"model": "gemini-2.0", "external_client": external_client, "tracing": True}
    user_session.set("run_config", run_config)
    agent = Agent(name="Assistant", instructions="You are a helpful assistant")
    user_session.set("agent", agent)
    await cli.message(content="Welcome to the AI Assistant! How can I help you?").send()

@on_message
async def main(message):
    run_config = cast(dict, user_session.get("run_config"))
    agent = cast(Agent, user_session.get("agent"))
    chat_history = user_session.get("chat_history", [])
    chat_history.append({"role": "user", "content": message})

    try:
        response = Runner.run_sync(agent, message, config=run_config)
        chat_history.append({"role": "assistant", "content": response.final_output})
        user_session.set("chat_history", chat_history)
        await cli.message(content=response.final_output).send()
    except Exception as e:
        await cli.message(content=f"Error: {str(e)}").send()
```

Explanation: This code initializes a chatbot with Chainlit. The @on_chat_start function sets up the Gemini API client, agent, and session configuration. The @on_message function processes user input, queries the LLM, and displays the response. The chat history ensures context is preserved for multi-turn conversations.

Why Its Important:

- Chatbots are a practical application of AI agents, widely used in industries like customer service, education, and healthcare.
- They prepare students for Quarter 4's focus on building advanced conversational AI systems, combining technical skills with user experience design.

3.3 Open Router

***Definition and Purpose**: Streaming refers to the process of displaying AI-generated responses in real-time, as they are produced by the LLM, rather than waiting for the entire response to be generated. In Class 14, streaming enhances the user experience of the Chainlit-based chatbot by showing response chunks (or tokens) as they are received, making interactions feel more dynamic and responsive.*

Key Features:

- **Real-Time Output:** Responses are displayed gradually, token by token, reducing perceived latency for users.
- **Asynchronous Processing:** Streaming uses asynchronous programming (e.g., async for loops) to handle incoming data efficiently.
- **Improved User Experience**: By showing partial responses immediately, streaming avoids long wait times, especially for complex queries.

How It Works:

- Instead of using Runner.run_sync (which waits for the full response), the streaming version uses Runner.run_stream to process LLM output in chunks.
- Chainlit's stream_token method updates the UI with each chunk, creating a typewriter-like effect as the response appears.

- The chat history is updated with the final response for context in subsequent interactions.

Practical Application:

- In today's class, students implemented streaming in their chatbot to display responses incrementally. This was demonstrated using the Runner.run_stream method.

Example (Streaming version):

```python
@on_message
async def main(message):
    run_config = cast(dict, user_session.get("run_config"))
    agent = cast(Agent, user_session.get("agent"))
    chat_history = user_session.get("chat_history", [])
    chat_history.append({"role": "user", "content": message})

    try:
        async for event in Runner.run_stream(agent, message, config=run_config):
            if event.event_type == "response_event":
                await cli.message(content=event.data.delta).stream_token()
        chat_history.append({"role": "assistant", "content": event.data.delta})
        user_session.set("chat_history", chat_history)
    except Exception as e:
        await cli.message(content=f"Error: {str(e)}").send()
```

Explanation: The Runner.run_stream method processes the LLM's output as a stream of events. When an event of type response_event is received, the stream_token method displays the delta (a chunk of the response) in the UI. The chat history is updated with the final response for context.

Why Its Important:

- Streaming enhances the interactivity of AI applications, making them more engaging and user-friendly.
- It's a critical skill for building real-world conversational systems, where low latency and responsiveness are key to user satisfaction.

3.4 Tools

***Definition and Purpose**: Streaming refers to the process of displaying AI-generated responses in real-time, as they are produced by the LLM, rather than waiting for the entire response to be generated. In Class 14, streaming enhances the user experience of the Chainlit-based chatbot by showing response chunks (or tokens) as they are received, making interactions feel more dynamic and responsive.*

Key Features:

- **Function Calling:** Tools are implemented as Python functions that the agent can call based on user queries.
- **Custom and Host Tools:**

  → **Custom Tools:** Developer-defined functions (e.g., fetching weather data via an API).
  → **Host Tools:** Pre-built tools provided by platforms like Open AI (e.g., web search).

- **Integration with Agents**: Tools are registered with the agent, which decides when to invoke them based on the query and instructions.

How It Works:

- Tools are defined using the function_tool decorator from the openai-agents library, which specifies the function's purpose and parameters.
- The agent is configured with a list of tools and instructions to use them for specific tasks.

- When a user query matches a tool's purpose (e.g., "What's the weather in Karachi?"), the agent calls the tool instead of relying on the LLM's knowledge.

Practical Application:

- In today's class, students created a custom tool to fetch weather data using an API, integrated with their chatbot.

Example:

```python
from openai_agents import Agent, Runner, function_tool
import requests

@function_tool
def get_weather(location: str, unit: str = "celsius") -> str:
    """Fetches weather for a given location."""
    api_key = os.getenv("WEATHER_API_KEY")
    url = f"http://api.weatherapi.com/v1/current.json?key={api_key}&q={location}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        return f"Weather in {location} is {data['current']['condition']['text']}."
    return "Unable to fetch weather data."

load_dotenv()
agent = Agent(name="Assistant", instructions="Use the get_weather tool for weather queries.", tools=[get_weather])
result = Runner.run_sync(agent, "What is the weather in Karachi?", config={"model": "gemini-2.0"})
print(result.final_output)
# Output: Weather in Karachi is sunny.
```

Explanation: The get_weather function is decorated with @function_tool, making it available to the agent. When the user asks about the weather, the agent calls the function, which queries the Weather API and returns the result. The LLM does not guess the weather; instead, the tool provides accurate, real-time data.

Why Its Important:

- Tools enable agents to overcome LLM limitations, such as outdated or missing data, making them more practical for real-world applications.
- They are essential for building versatile AI systems that can handle diverse tasks, from data retrieval to automation.

3.5 Tool Calling

***Definition and Purpose**: Tool calling is the mechanism by which an AI agent decides to invoke a tool to handle a specific user query. It involves the agent interpreting the query, matching it to the appropriate tool based on instructions, and executing the tool to generate a response. In Class 14, tool calling was demonstrated as a way to extend the chatbot's functionality beyond LLM-generated responses.*

Key Features:

- **Agent Instructions:** The agent is given instructions (e.g., "Use the get_weather tool for weather queries") to guide tool selection.
- **Function Matching:** The agent uses natural language understanding to determine when a tool is relevant (e.g., recognizing "weather" in a query).
- **Execution And Response**: The agent calls the tool, passes the necessary parameters (e.g., location), and integrates the tool's output into the response.

How It Works:

- The agent is initialized with a list of tools and instructions specifying when to use them.
- When a user query is received, the agent's logic (powered by the LLM and openai-agents) evaluates whether a tool is needed.
- If a tool is relevant, the agent calls it, passes the required inputs, and returns the tool's output to the user. If no tool is needed, the LLM generates the response directly.

Practical Application:

- In the weather tool example above, tool calling allows the agent to recognize a weather-related query and invoke the get_weather function. This ensures accurate, real-time data instead of relying on the LLM's potentially outdated knowledge.
- **Example Scenario**: A user asks, "What's the weather in Karachi?" The agent identifies the query as weather-related, calls the get_weather tool with location="Karachi", and returns "Weather in Karachi is sunny."

Why Its Important:

- Tool calling enhances the agent's autonomy and versatility, allowing it to handle specialized tasks that LLMs cannot perform alone.
- It prepares students for building complex AI systems in Quarter 4, where agents must integrate multiple tools for tasks like data analysis, automation, or real-time information retrieval.

---

4. Practical Coding and Debugging

- **Environment Setup**: Students were instructed to create a UV project for the Chainlit-based chatbot, setting up a virtual environment and installing dependencies like openai-agents, chainlit, and python-dotenv. A .env file was used to store the Gemini API key securely, ensuring proper configuration for LLM integration.
- **Debugging Example**: Common issues, such as missing API keys or incorrect Chainlit configurations, were resolved by verifying the .env file setup and using os.getenv to load keys. Errors in streaming responses were debugged by checking event types in the Runner.run_stream loop.
- **Validation Logic**: Students were emphasized to avoid exposing sensitive data, such as API keys, in logs or console outputs. Secure coding practices were reinforced by validating inputs and using tracing for observability.

---

5. Key Insights and Takeaways

- **AI Agent Development**: Mastery of Chainlit, Chatbot, Streaming, and Tools is essential for building practical AI applications in Quarter 4.
- **Self Learning**: Students must adopt a self-directed learning approach using resources like the Panaversity repository to prepare for advanced topics.
- **Industry Relevance**: Skills in AI agents and tools are in high demand, offering career opportunities in global markets when paired with strong communication skills.
- **Professional Coding Practices**: Emphasize observability (e.g., Open AI tracing), proper prompt engineering, and structured code organization to become professional developers.

---

6. Class Resources and Assignments

- **Today's Class Code**
  Students are encouraged to experiment with the provided examples to deepen understanding.
- **Complete Previous Assignments**:
  Ensure that the previous assignments are completed and pushed to GitHub, and submit the link via the assignment submission form.

Important Reminder

- Use documentation and tools like NotebookLM for test preparation, focusing on practical coding skills.
- All Agents projects must be on GitHub.

- Stay active on Discord for class announcements and review the Panaversity repository for the syllabus.

Final Words

Your responsibility is to learn actively, participate confidently, and practice consistently. Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

Allah Hafiz – See you in the Next Class!

Repo Link: GIAIC-Sat-Afternoon

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. You can go through each class code and its full detailed summary without any hesitation, and it will also help you prepare well for your test.

Thank You for Reading!

Hope you understood Class 14 well.

"True Education doesn’t fill your mind, it frees it – to question, to create, and to change the world." – *Sahir Ahmed Sheikh*